AADK 13: Task 1 — Accessibility Audit + Improvement Plan.

Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – **Session 13**

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

Accessibility Audit & Improvement Plan

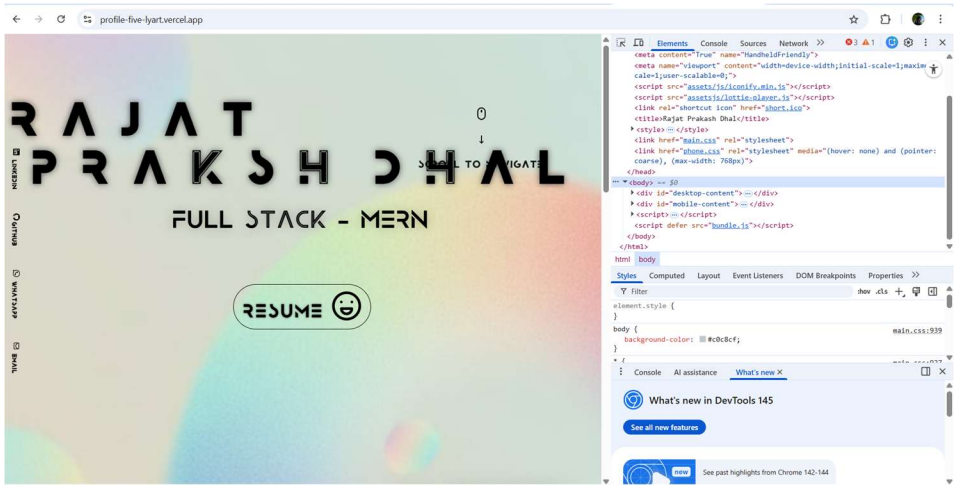
Screen Selected: Superhero List Screen (Jetpack Compose UI)

1. Selected UI Screen

The selected screen for the accessibility audit is the **Superhero List Screen** implemented in Jetpack Compose. The screen displays a scrollable list of superheroes with the following components:

- Superhero image (avatar)
- Name and description text
- Favourite button icon
- Card layout with click interaction

The screen was previously developed as part of the animated themed list feature.



---

2. Accessibility Audit Checklist

Accessibility Criteria	Current Status	Evidence / Observation
Content Description for Images	Need Improvement	Avatar images currently lack contentDescription, making them unreadable for screen readers.
Touch Target Size	Pass	Buttons and icons mostly meet the recommended **48dp minimum size**.
Colour Contrast	Need Improvement	Some theme combinations use light grey text on white background, reducing readability.
Motion Reduction Support	Need Improvement	List animations always play; no check for system "Reduce Motion" preference.
Focus Order & Navigation	Pass	Compose semantics automatically follows logical order in the list.
Text Size Scalability	Need Improvement	Some text containers use fixed height which may clip text when system font size increases.

---

3. Accessibility Improvement Plan

1. Add Content Descriptions for Images

Priority: High

Problem: Screen readers cannot describe superhero avatars.

Fix Implementation:

```
Image(
    painter = painterResource(hero.imageRes),
    contentDescription = "${hero.name} avatar"
)
```

Decorative images will use:

contentDescription = null

---

2. Ensure Minimum Touch Target Size

Priority: Medium

Fix to guarantee accessibility-compliant tap targets.

Modifier.sizeIn(minHeight = 48.dp, minWidth = 48.dp)

Applied to buttons and clickable icons.

---

3. Improve Colour Contrast

Priority: High

Switch text colours to Material theme colours that meet **WCAG AA contrast ratio (≥ 4.5:1)**.

Example:

```
Text(
    text = hero.name,
    color = MaterialTheme.colorScheme.onSurface
)
```

Use colorScheme tokens instead of hardcoded colors.

---

4. Support Motion Reduction

Priority: Medium

Animations should respect the user's **Reduce Motion** system preference.

Example strategy:

```
if (!reduceMotionEnabled) {
    animateItemPlacement()
}
```

This ensures accessibility for users sensitive to motion.

---

5. Improve Text Scalability

Priority: Medium

Replace fixed heights with flexible layouts.

Example:

Modifier.wrapContentHeight()

Use scalable typography:

style = MaterialTheme.typography.bodyLarge

This ensures layout adapts when system font size increases.

---

4. Accessibility Testing Integration

Accessibility tests will be implemented using **Jetpack Compose UI Testing APIs**.

Library / APIs Used

- androidx.compose.ui.test
- SemanticsMatcher
- semantics() modifier

Test 1 — Verify Image Accessibility

composeTestRule

.onNodeWithContentDescription("Superman avatar")

.assertIsDisplayed()

This ensures images include meaningful descriptions.

---

composeTestRule

.onRoot()

.printToLog("AccessibilityTree")

This helps verify the accessibility structure and detect missing semantics.

Test 3 — Button Accessibility

composeTestRule

.onNodeWithText("Favourite")

.assertHasClickAction()

Ensures interactive elements are properly accessible.

---

5. Reflection

Accessibility is critical because modern applications must be usable by **all users, including people with visual, motor, or cognitive impairments**. Designing accessible interfaces improves usability, readability, and interaction quality for everyone, not just users with disabilities.

Planning accessibility from the start helps avoid expensive redesigns later and ensures the UI structure supports screen readers, scalable text, and proper interaction patterns.

During the audit, one challenge discovered was **missing semantic descriptions for images**, which is easy to overlook but significantly affects screen reader usability.